



Guia Completo: Spring Boot no Docker e Kubernetes

Este guia abrangente apresenta um passo a passo detalhado para criar, containerizar e implantar uma aplicação Spring Boot em ambiente Kubernetes. Desenvolvedores Java encontrarão aqui uma jornada completa desde a criação do projeto até o deployment em produção, com explicações claras e práticas recomendadas.

Assumimos conhecimento básico de Java, Docker e Kubernetes, além de ter as ferramentas essenciais instaladas: JDK 17+, Maven/Gradle, Docker Desktop e um cluster K8s (Minikube ou cloud).

Spring Boot

Database

Preparação do Ambiente de Desenvolvimento



Java JDK 17+

Instale via Oracle ou OpenJDK para garantir compatibilidade com Spring Boot 3.x

- Download do site oficial
- Configuração das variáveis de ambiente
- Verificação com `java -version`



Spring Initializr

Ferramenta web para scaffolding de projetos Spring Boot

- Interface intuitiva e configurável
- Seleção de dependências automatizada
- Geração de estrutura padrão



Docker Desktop

Plataforma de containerização essencial para empacotamento da aplicação

- Interface gráfica amigável
- Suporte a multi-stage builds
- Integração com registries



Kubernetes

Orquestrador de containers para deployment e escalabilidade

- Minikube para desenvolvimento local
- Clusters gerenciados (EKS/GKE/AKS)
- `kubectl` como ferramenta de linha de comando

Antes de iniciar o desenvolvimento, certifique-se de que todas essas ferramentas estão instaladas e funcionando corretamente. Execute comandos de verificação como `java -version`, `docker --version` e `kubectl version` para confirmar a instalação.

Criando o Projeto Spring Boot

01

Configuração no Spring Initializr

Acesse start.spring.io e configure:

- Project: Maven (recomendado)
- Language: Java
- Spring Boot: Versão 3.3.x (mais recente)
- Group: com.example
- Artifact: demo
- Dependencies: Spring Web para APIs REST

02

Download e Configuração

Após gerar o projeto:

- Baixe o arquivo ZIP gerado
- Extraia em diretório de trabalho
- Abra na sua IDE preferida (IntelliJ, Eclipse, VS Code)
- Aguarde a sincronização das dependências Maven

03

Estrutura do Projeto

Explore a estrutura gerada automaticamente:

- src/main/java: Código fonte principal
- src/test/java: Testes automatizados
- pom.xml: Configuração de dependências Maven
- application.properties: Configurações da aplicação

Desenvolvendo o Primeiro Endpoint

Criando o Controller

Para demonstrar o funcionamento da aplicação, criaremos um endpoint simples que retorna uma mensagem de boas-vindas. Esta abordagem permite verificar rapidamente se a aplicação está funcionando após o deployment.

```
package com.example.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Olá, mundo! Spring Boot rodando.";
    }

    @GetMapping("/health")
    public String health() {
        return "Aplicação funcionando perfeitamente!";
    }
}
```

Benefícios do Endpoint

- Verificação rápida do funcionamento
- Health check para Kubernetes
- Base para expansão da API
- Facilita debugging e monitoramento

Teste Local

Execute o comando no terminal:

```
./mvnw spring-boot:run
```

Acesse <http://localhost:8080/hello> para verificar a resposta.

Containerização com Docker

Criando o Dockerfile Otimizado

O Dockerfile utiliza estratégia multi-stage para otimizar o tamanho final da imagem e melhorar a segurança. A primeira etapa compila a aplicação, enquanto a segunda cria uma imagem mínima apenas com o runtime necessário.

1

Etapa de Build

```
# Etapa 1: Build
FROM maven:3.8.6-openjdk-17 AS build
WORKDIR /app
COPY . .
RUN mvn clean package -DskipTests
```

Esta etapa utiliza uma imagem completa do Maven para compilar o código fonte e gerar o JAR executável.

2

Etapa de Runtime

```
# Etapa 2: Runtime
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY --from=build /app/target/demo-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

A segunda etapa usa uma imagem mínima com apenas o JRE, resultando em container mais leve e seguro.

1

Build da Imagem

```
docker build -t meu-spring-boot-app:latest .
```

2

Teste Local

```
docker run -p 8080:8080 meu-spring-boot-app:latest
```

3

Push para Registry

```
docker push seuusuario/meu-spring-boot-app:latest
```

Preparando para Kubernetes

Antes de fazer o deployment no Kubernetes, é essencial entender os conceitos fundamentais e preparar os arquivos de configuração. O Kubernetes gerencia nossa aplicação através de recursos declarativos que definem o estado desejado do sistema.



Deployment

Define como os pods da aplicação devem ser criados e gerenciados. Especifica a imagem Docker, número de réplicas, recursos necessários e estratégias de atualização. É o recurso principal que garante que nossa aplicação esteja sempre executando conforme especificado.



Service

Expõe a aplicação para tráfego de rede, criando um ponto de acesso estável. Abstrai os pods individuais e fornece balanceamento de carga automático. Essencial para permitir comunicação entre componentes e acesso externo.



Cluster

Ambiente onde todos os recursos são executados. Pode ser local (Minikube) ou em nuvem (EKS/GKE/AKS). Fornece alta disponibilidade, escalabilidade e gerenciamento centralizado de recursos computacionais.

- ❏ **Dica Importante:** Para ambientes de produção, considere configurar recursos como ConfigMaps para variáveis de ambiente, Secrets para informações sensíveis e PersistentVolumes para armazenamento de dados.

Configuração do Deployment

Arquivo deployment.yaml

O Deployment é o recurso central que gerencia o ciclo de vida dos pods da nossa aplicação. Ele garante que o número especificado de instâncias esteja sempre executando e facilita atualizações sem downtime.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: spring-boot-demo
  labels:
    app: spring-boot-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: spring-boot-demo
  template:
    metadata:
      labels:
        app: spring-boot-demo
    spec:
      containers:
        - name: app
          image: meu-spring-boot-app:latest
          ports:
            - containerPort: 8080
          resources:
            requests:
              memory: "256Mi"
              cpu: "250m"
            limits:
              memory: "512Mi"
              cpu: "500m"
          livenessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 30
            periodSeconds: 10
```

Réplicas

Número de pods que serão executados simultaneamente, garantindo alta disponibilidade e distribuição de carga.

Recursos

Limites de CPU e memória para cada pod, essenciais para planejamento de capacidade e estabilidade.

Health Checks

Probes que monitoram a saúde da aplicação e reiniciam containers com problemas automaticamente.

Configuração do Service

Arquivo service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: spring-boot-service
  labels:
    app: spring-boot-demo
spec:
  selector:
    app: spring-boot-demo
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
      name: http
  type: LoadBalancer
```

O Service atua como um proxy reverso, direcionando o tráfego para os pods corretos e fornecendo balanceamento de carga automático entre as instâncias disponíveis.

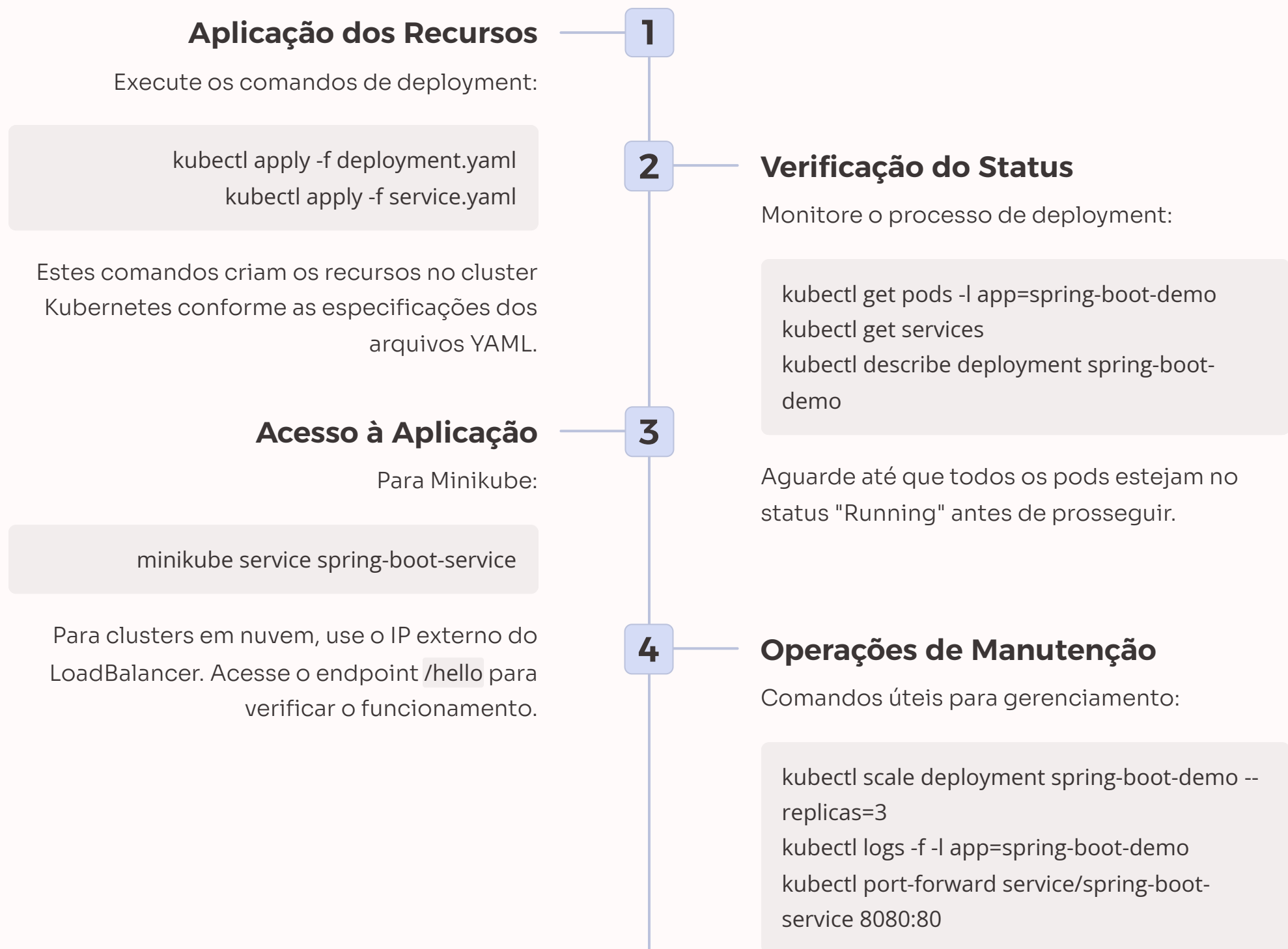
Tipos de Service

- **ClusterIP:** Acesso interno apenas
- **NodePort:** Expõe em porta específica do nó
- **LoadBalancer:** Integração com load balancers da nuvem
- **ExternalName:** Mapeia para serviços externos

Configuração de Portas

A configuração mapeia a porta 80 (externa) para a porta 8080 (container), permitindo acesso padronizado via HTTP. O `targetPort` deve corresponder à porta exposta pela aplicação Spring Boot.

Deploy e Monitoramento



99.9%

Disponibilidade

Alta disponibilidade com múltiplas réplicas

5x

Escalabilidade

Facilidade de escalar horizontalmente

30s

Deploy

Tempo médio para rollout de atualizações

Práticas Recomendadas e Próximos Passos

Segurança

- Implementar HTTPS com certificados TLS
- Usar Secrets para dados sensíveis
- Configurar Network Policies
- Aplicar RBAC (Role-Based Access Control)
- Escanear imagens por vulnerabilidades

Monitoramento

- Integrar Prometheus e Grafana
- Configurar alertas inteligentes
- Implementar distributed tracing
- Usar métricas customizadas da aplicação
- Log aggregation com ELK Stack

CI/CD

- Automatizar builds com GitHub Actions
- Pipeline de testes automatizados
- Deploy automatizado por ambiente
- Rollback automático em falhas
- Code quality gates

Evolução

- Adicionar banco de dados persistente
- Implementar cache distribuído
- Microserviços architecture
- Service mesh (Istio/Linkerd)
- Observabilidade avançada

Considerações para Produção

Este guia fornece uma base sólida para desenvolvimento e deployment de aplicações Spring Boot em Kubernetes. Para ambientes de produção, considere implementar estratégias avançadas como blue-green deployments, canary releases, auto-scaling baseado em métricas personalizadas e disaster recovery.

A jornada de aprendizado continua com tópicos como service mesh, observabilidade avançada e arquitetura de microserviços. Mantenha-se atualizado com as melhores práticas da comunidade e sempre priorize segurança e performance em suas implementações.

- 📌 **Próximo Nível:** Explore ferramentas como Helm para gerenciamento de pacotes Kubernetes, Skaffold para desenvolvimento iterativo, e Tekton para pipelines nativos do Kubernetes.